

UNIVERSIDAD DE LAS AMÉRICAS PUEBLA

Departamento de Computación, Electrónica y Mecatrónica

Reporte de práctica

Práctica 3: Memorias ROM y RAM para la Tarjeta PYNQ Z1

Materia	P26-LIS2022-1 Arquitecturas Computacionales
Profesor	Dr. Omar Guillén Fernández
Sección	2
Periodo	Primavera 2026

Integrante

Robbie Nicolás Curioso de Salazar 186028 robbie.curiosodr@udlap.mx



1. Introducción

Las memorias son uno de los bloques esenciales de cualquier sistema digital: almacenan tanto las instrucciones que ejecuta un procesador como los datos que éste procesa. En esta práctica se diseñaron e implementaron, en VHDL, dos tipos de memorias síncronas pensadas para sintetizarse sobre la tarjeta PYNQ Z1, que está basada en el SoC Zynq 7000 (FPGA + procesador ARM): una memoria ROM con datos de cumpleaños, y una memoria RAM para almacenar fechas de nacimiento.

Adicionalmente, se planteó el uso de memorias ROM como almacén de programas escritos en lenguaje ensamblador RISC-V, como continuación de la Tarea 3. Para ello se discuten tres pequeños programas: el cálculo de $(a + b) \cdot c$, el factorial de cinco y la detección de paridad de un entero, junto con su codificación binaria en RV32I que se grabaría en cada ROM.

La verificación funcional se realizó mediante simulación en *Active-HDL* con un banco de pruebas que ejercita la lectura de la ROM de cumpleaños y la escritura/lectura de la RAM de fechas.

2. Objetivos

- Implementar en VHDL una memoria ROM síncrona inicializada con datos en código, configurable mediante *generics* de ancho de dirección y ancho de dato.
- Implementar una memoria RAM síncrona de un puerto, con escritura controlada por *we* y lectura registrada.
- Cargar la ROM con cumpleaños de compañeros del curso en formato BCD (DDMM) y la RAM con fechas de nacimiento de 32 bits en formato DDMMYYYY.
- Discutir el uso de memorias ROM como almacén de programas RV32I, presentando el ensamblador y la codificación hexadecimal de los tres programas planteados en la Tarea 3.
- Validar el comportamiento síncrono de ambas memorias mediante simulación en Active-HDL y diagramas de tiempo.

3. Materiales y herramientas

- **Software:** Active-HDL Student Edition para compilación y simulación funcional.
- **Lenguaje:** VHDL, con uso de las bibliotecas `ieee.std_logic_1164` y `ieee.numeric_std`.
- **Tecnología objetivo:** Zynq 7000 sobre tarjeta PYNQ Z1 (referencia para discutir la inferencia de bloques BRAM).
- **Documentación:** guía oficial de la práctica, documentos UG585 y UG953 de Xilinx, y la especificación RV32I de RISC-V para la sección de Tarea 3.

4. Marco teórico

4.1. Memoria ROM

Una ROM (*Read-Only Memory*) es una memoria de solo lectura cuyo contenido queda fijado en el momento del diseño. En VHDL se modela como un arreglo constante indexado por una dirección. Para que el sintetizador la reconozca como memoria distribuida o como bloque BRAM dentro de la FPGA, la lectura se realiza dentro de un proceso síncrono al reloj, lo cual introduce un retardo de un ciclo entre la presentación de la dirección y la aparición del dato en la salida.

4.2. Memoria RAM de un puerto

Una RAM síncrona de un puerto admite escritura y lectura sobre la misma dirección, sincronizadas con el flanco de subida del reloj. Cuando la señal de habilitación `we` se encuentra en alto, el dato presentado en `data_in` se almacena en la posición indicada; en cualquier otro caso, la salida `data_out` entrega el dato de la dirección registrada en el ciclo anterior. Esta convención es la que recomienda Xilinx para inferir BRAMs en la familia 7 Series.

4.3. Bloques BRAM en el Zynq 7000

El Zynq 7000 que monta la PYNQ Z1 cuenta con bloques de memoria dedicados (BRAM) de 36 Kb cada uno, distribuidos en columnas dentro del tejido de la FPGA. Cuando un diseño VHDL describe una memoria síncrona con un patrón estándar de lectura/escritura,

el sintetizador (Vivado) infiere automáticamente uno o varios BRAMs para implementarla. Para profundidades pequeñas (decenas de palabras), el sintetizador puede optar por lógica distribuida (LUTs configurados como pequeñas memorias) en lugar de un BRAM completo.

4.4. Codificación de instrucciones RV32I

Cada instrucción RISC-V de 32 bits se compone de campos fijos. Las instrucciones de tipo I, como ADDI o ANDI, tienen el formato `imm[11:0] | rs1 | funct3 | rd | opcode`. Las instrucciones de tipo R, como ADD, agregan los campos `funct7` y `rs2`. Para los corrimientos con inmediato (SLLI), el campo inmediato se interpreta como una cantidad de cinco bits ubicada en `imm[4:0]`, junto con `funct7=0000000`.

5. Diseño y procedimiento

5.1. Memoria ROM de cumpleaños

Se describió la entidad `rom_cumpleanos` con dos genéricos: `addr_width` y `data_width`. Para la práctica se fijó `addr_width=4` (16 posiciones) y `data_width=16` (suficientes para empaquetar día y mes en BCD). El contenido de la memoria se declara como una constante de tipo arreglo, donde cada entrada lleva el formato `0xDDMM`, por ejemplo `0x1505` representa el 15 de mayo. La lectura se hace dentro de un proceso síncrono al reloj que copia el dato direccionado a una señal interna, la cual luego se propaga a la salida `data_out`.

5.2. Memoria RAM de fechas de nacimiento

La entidad `ram_fechas` sigue una estructura similar pero con `data_width=32` para almacenar fechas en formato `0xDDMMYYYY`. La memoria se modela con una señal interna inicializada a cero (lo cual permite verificar su estado inicial en simulación). Dentro del proceso síncrono se evalúa la señal `we`: si está en alto, se escribe el dato de entrada en la posición indicada; en cualquier caso, la dirección se registra en una señal auxiliar `read_addr`, y la salida combinacional `data_out` se obtiene leyendo la posición indicada por `read_addr`. Este patrón es estándar para inferir BRAMs y produce un retardo de un ciclo en la lectura.

5.3. Banco de pruebas

El testbench `tb_memorias` instancia tanto la ROM de cumpleaños como la RAM de fechas, genera un reloj con período de 10.000 ns y aplica los siguientes estímulos:

1. Recorrido completo de la ROM, leyendo las direcciones 0 a 15 con un ciclo por dirección.
2. Cinco escrituras en la RAM con `we=1`: la fecha `0x15052003` en la posición 0, `0x22082002` en 1, `0x07122003` en 5, `0x14112002` en 10 y `0x30012004` en 15.
3. Recorrido completo de la RAM con `we=0`, leyendo las posiciones 0 a 15 para verificar tanto las direcciones escritas como las que permanecen en cero.

5.4. Flujo en Active-HDL

Para cada simulación se siguió el procedimiento habitual:

1. Crear el espacio de trabajo y agregar los archivos `rom_cumpleanos.vhd`, `ram_fechas.vhd` y `tb_memorias.vhd`.
2. Compilar primero las dos memorias y, finalmente, el banco de pruebas.
3. Inicializar la simulación con `tb_memorias` como nivel superior.
4. Agregar al *waveform* las ocho señales de interés (`clk`, `cump_addr`, `cump_data_out`, `ram_we`, `ram_addr`, `ram_data_in`, `ram_data_out`) y configurar el formato hexadecimal para las señales de datos y binario para las direcciones.
5. Ejecutar *Run For* con duración de 400.000 ns.

6. Resultados y discusión

La Figura 1 muestra la simulación completa, donde se aprecian las dos fases: la primera mitad corresponde al recorrido de la ROM de cumpleaños y la segunda mitad al ciclo de escritura y lectura de la RAM de fechas. En las subsecciones siguientes se analiza cada bloque por separado.

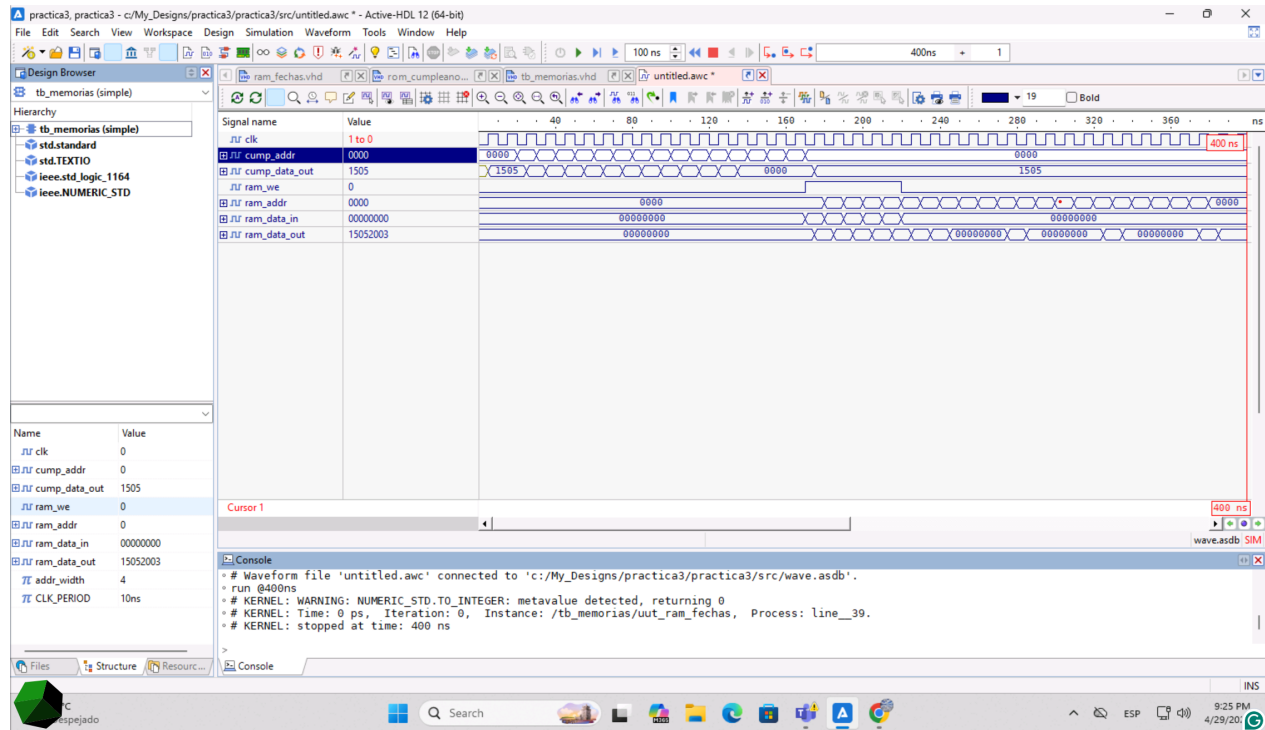


Figura 1: Vista panorámica de la simulación: lectura de la ROM seguida de escritura y lectura de la RAM.

6.1. ROM de cumpleaños

La Figura 2 muestra la lectura secuencial de las dieciséis direcciones de la ROM. Conforme la dirección `cump_addr` avanza de 0000 hasta 1111, la salida `cump_data_out` entrega el cumpleaños correspondiente con un ciclo de retraso, lo cual confirma que la ROM funciona como memoria síncrona.

Cuadro 1: Contenido de la ROM de cumpleaños (formato BCD: 0xDDMM).

Dirección	Valor (hex)	Fecha (DD/MM)
0	1505	15/05
1	2208	22/08
2	0712	07/12
3	3001	30/01
4	1411	14/11
5	0904	09/04
6	1806	18/06
7	0211	02/11
8	2503	25/03
9	1207	12/07
10	0509	05/09
11	2802	28/02
12–15	0000	–

Las posiciones 12 a 15 se dejaron en cero para reflejar el comportamiento de una ROM con menos entradas que su capacidad nominal: las direcciones no inicializadas explícitamente devuelven el valor por defecto. Esto es consistente con la práctica común de declarar una ROM más grande que el contenido para dejar espacio a futuras adiciones.

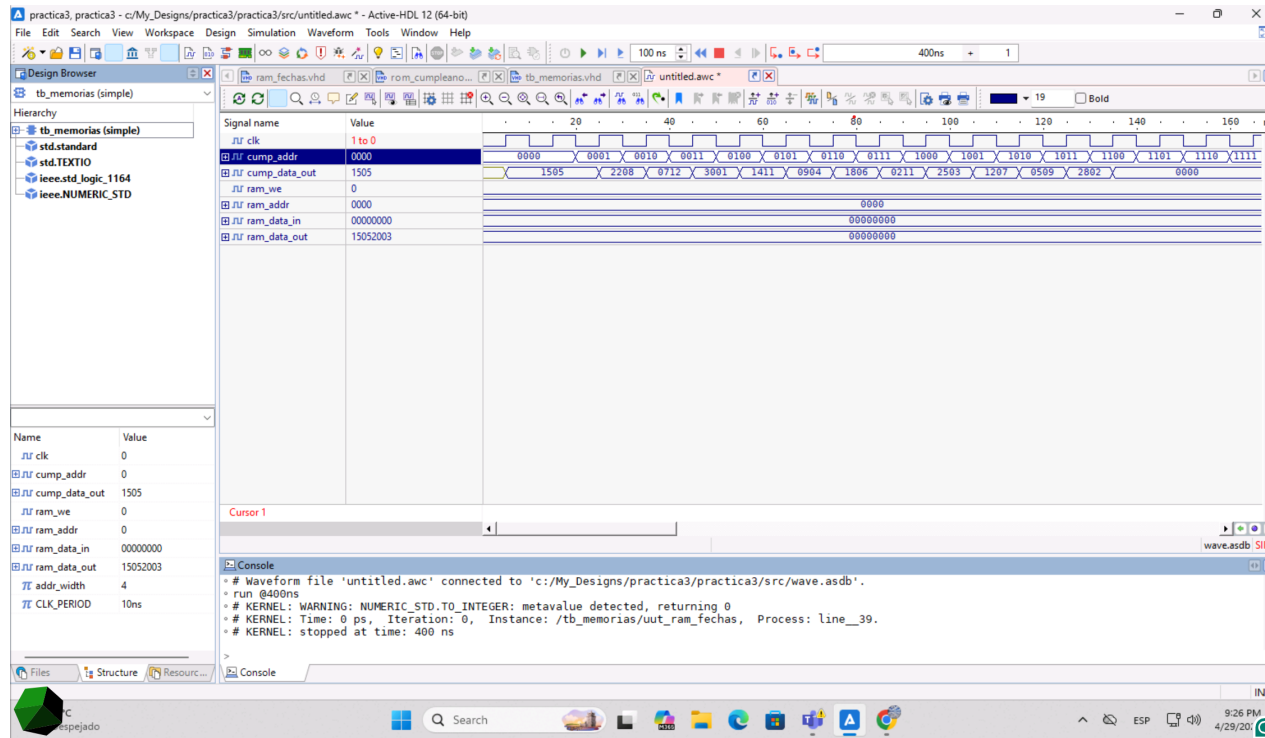


Figura 2: Lectura de las dieciséis direcciones de la ROM de cumpleaños.

6.2. RAM de fechas de nacimiento

La Figura 3 concentra las dos fases de la RAM. Durante la primera región la señal **ram_we** permanece en alto y se aplican cinco vectores de entrada: 0x15052003, 0x22082002, 0x07122003, 0x14112002 y 0x30012004, dirigidos a las posiciones 0, 1, 5, 10 y 15 respectivamente. En la segunda región **ram_we** baja a cero y la dirección recorre las dieciséis posiciones; la salida **ram_data_out** entrega los valores escritos en las cinco direcciones modificadas y devuelve cero en las demás, confirmando tanto la inicialización a cero como la persistencia de los datos almacenados.

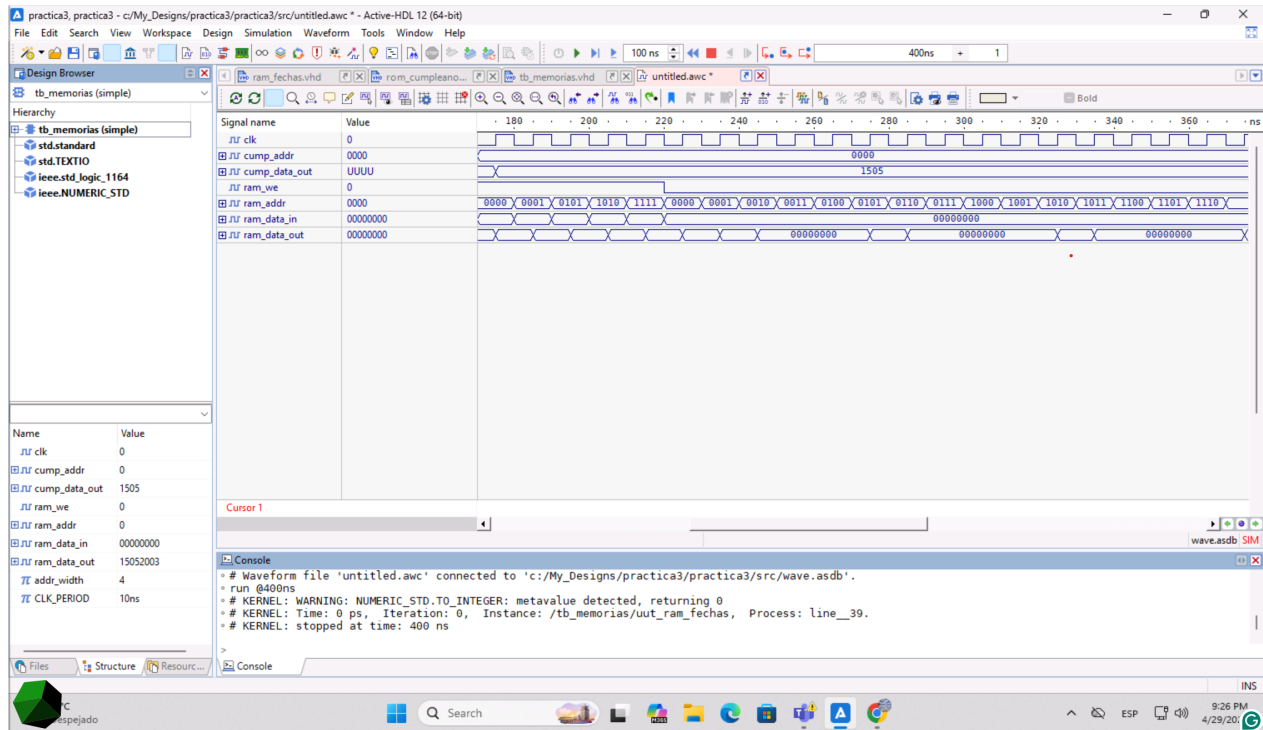


Figura 3: Operación de la RAM de fechas: ciclo de escritura seguido de lectura completa.

Es importante notar el retardo de un ciclo entre la presentación de una dirección y la aparición del dato en la salida. Este retardo es propio de la arquitectura síncrona y proviene del registro intermedio (`read_addr`) que se utiliza para que el sintetizador infiera correctamente un BRAM. En un sistema completo, este retardo debe tenerse en cuenta al conectar la RAM a una unidad de control: la señal de validez del dato leído está disponible un ciclo después de la dirección.

6.3. ROMs de la Tarea 3

Adicionalmente al uso de la ROM como almacén de datos, se planteó como ejercicio cargar tres pequeños programas escritos en RV32I dentro de memorias ROM independientes. Estas ROMs no se simulan en este banco de pruebas, pero se documenta su contenido para complementar la discusión.

6.3.1. ROM de $(a + b) \cdot c$

El programa calcula $(5 + 5) \cdot 4 = 40$ usando un corrimiento a la izquierda en lugar de una multiplicación, ya que el conjunto base RV32I no incluye MUL. La secuencia de cinco

instrucciones se muestra en la Tabla 2.

Cuadro 2: Programa RV32I para $(a + b) \cdot c$ con $a = 5$, $b = 5$, $c = 4$.

Dirección	Ensamblador	Hex	Significado
0	addi x1, x0, 5	0x00500093	$x_1 \leftarrow 5$
1	addi x2, x0, 5	0x00500113	$x_2 \leftarrow 5$
2	addi x3, x0, 4	0x00400193	$x_3 \leftarrow 4$
3	add x4, x1, x2	0x00208233	$x_4 \leftarrow x_1 + x_2 = 10$
4	slli x5, x4, 2	0x00221293	$x_5 \leftarrow x_4 \ll 2 = 40$

El resultado final, $x_5 = 40$, queda almacenado en el registro **x5**. El uso del corrimiento a la izquierda por dos posiciones equivale a multiplicar por 4, lo cual aprovecha el hecho de que c es una potencia de 2.

6.3.2. ROM del factorial de cinco

El cálculo de $5! = 120$ se realizó descomponiendo la operación en corrimientos y una suma, dado que $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 5 \cdot 24$ y $24 = 16 + 8 = 5 \cdot 4 + 5 \cdot 4 / \dots$. La secuencia adoptada calcula primero $5 \cdot 4 = 20$ con un corrimiento, luego $20 \cdot 3$ como $20 \cdot 2 + 20$ y finalmente multiplica por 2 con un último corrimiento.

Cuadro 3: Programa RV32I para $5!$ usando corrimientos y una suma.

Dirección	Ensamblador	Hex	Significado
0	addi x1, x0, 5	0x00500093	$x_1 \leftarrow 5$
1	slli x4, x1, 2	0x00209213	$x_4 \leftarrow 5 \cdot 4 = 20$
2	slli x5, x4, 1	0x00121293	$x_5 \leftarrow 20 \cdot 2 = 40$
3	add x5, x5, x4	0x004282B3	$x_5 \leftarrow 40 + 20 = 60$
4	slli x6, x5, 1	0x00129313	$x_6 \leftarrow 60 \cdot 2 = 120$

El resultado, $x_6 = 120$, coincide con $5!$. Este enfoque ilustra cómo, sin la extensión M de RISC-V, las multiplicaciones por constantes cuyos factores son potencias cercanas a 2 pueden expresarse como combinaciones de corrimientos y sumas.

6.3.3. ROM de paridad

El programa de detección de paridad explota el hecho de que el bit menos significativo de un entero indica si es par o impar: tomar la operación AND con `0x00000001` aísla ese bit. La secuencia consta de dos instrucciones, mostradas en la Tabla 4.

Cuadro 4: Programa RV32I para detección de paridad con $x = 5$.

Dirección	Ensamblador	Hex	Significado
0	<code>addi x1, x0, 5</code>	<code>0x00500093</code>	$x_1 \leftarrow 5$
1	<code>andi x2, x1, 1</code>	<code>0x0010F113</code>	$x_2 \leftarrow x_1 \text{ AND } 1 = 1$

El resultado en `x2` es uno, lo que indica que el valor original es impar. Si en lugar de cinco se hubiera cargado un número par, `x2` habría quedado en cero.

7. Buenas prácticas

Durante el desarrollo se mantuvieron criterios que ayudan a obtener una síntesis predecible: las memorias se describieron con procesos síncronos puros (sin lógica combinatorial escondida), la lectura se registró antes de salir hacia el puerto de salida y se evitó usar tipos no estándar como `std_logic_arith`. La inicialización explícita de la RAM a cero permite que el comportamiento en simulación coincida con el de una RAM real recién encendida. En el banco de pruebas, los estímulos se aplican siempre alineados al período del reloj para que las transiciones queden visibles en el *waveform*, lo que simplifica la depuración.

8. Conclusiones

Se diseñaron y verificaron en VHDL una ROM de 16 bits para almacenar cumpleaños y una RAM de 32 bits para guardar fechas de nacimiento, ambas siguiendo los patrones recomendados por Xilinx para inferir bloques BRAM en la PYNQ Z1. La simulación en Active-HDL mostró que la ROM entrega correctamente los valores cargados como constante y que la RAM admite escrituras controladas por `we`, manteniendo el contenido entre operaciones de lectura. El retardo de un ciclo entre dirección y dato se confirmó como característica esperada de las memorias síncronas y debe considerarse al integrarlas en una unidad de control.

La discusión de las tres ROMs de la Tarea 3 reforzó la idea de que una memoria de instrucciones es funcionalmente una ROM cuyo contenido está fijo en tiempo de diseño: lo único que cambia entre uno y otro programa es el conjunto de instrucciones precargadas. La codificación manual de los programas también permitió verificar el formato binario de las instrucciones ADDI, ADD, SLLI y ANDI de RV32I.

Como trabajo futuro se contempla implementar también una RAM de doble puerto siguiendo la misma metodología, así como sintetizar las memorias en la PYNQ Z1 para confirmar la inferencia de BRAMs y medir el uso de recursos reportado por Vivado.

9. Referencias

Referencias

- [1] O. Guillén Fernández. *Práctica 3: Memorias ROM y RAM para la Tarjeta PYNQ Z1*. Universidad de las Américas Puebla.
- [2] Xilinx. *UG585: Zynq-7000 SoC Technical Reference Manual*. Disponible en: https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf
- [3] Xilinx. *UG953: 7 Series FPGAs Memory Resources*. Disponible en: https://docs.xilinx.com/v/u/en-US/ug473_7Series_Memory_Resources
- [4] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Specification*. Disponible en: <https://riscv.org/technical/specifications/>
- [5] P. J. Ashenden. *The Designer's Guide to VHDL*. Morgan Kaufmann, 2008.
- [6] Aldec. *Active-HDL Documentation*. Disponible en: https://www.aldec.com/en/products/fpga_simulation/active_hdl

Repositorio del proyecto

El código fuente VHDL de las memorias y del banco de pruebas, junto con las capturas de las formas de onda obtenidas en Active-HDL, se encuentran disponibles en el siguiente repositorio público de GitHub:

[https:](https://github.com/Robbienicur/Arquitecturas-Computacionales/tree/main/Practica-3)

[//github.com/Robbienicur/Arquitecturas-Computacionales/tree/main/Practica-3](https://github.com/Robbienicur/Arquitecturas-Computacionales/tree/main/Practica-3)

La carpeta Practica-3 del repositorio contiene los archivos `rom_cumpleanos.vhd`, `ram_fechas.vhd` y `tb_memorias.vhd`, las imágenes de la simulación y este mismo reporte en formato PDF.

